1. Install Docker and Create Dockerfiles to Containerize Applications

1.1 Introduction

Docker is an open-source platform used for **containerization**, allowing developers to package applications and their dependencies into portable containers.

Containers ensure that applications run consistently across different environments such as development, testing, and production.

Docker uses **Docker images** to create containers. These images are built using **Dockerfiles**, which define the instructions required to build the container.

1.2 What is Containerization?

Containerization is a lightweight virtualization technique that packages applications with all required dependencies into a single container.

Benefits of Containerization

- Consistent application environments
- Faster deployment
- Lightweight compared to virtual machines
- Easy scalability
- Efficient resource utilization

1.3 Overview of Docker Architecture

Docker architecture consists of the following components:

Component	Description
Docker Client	Interface used to run Docker commands
Docker Daemon	Background service that manages containers
Docker Images	Templates used to create containers
Docker Containers	Running instances of Docker images
Docker Hub	Cloud registry for Docker images

1.4 Installing Docker

Docker was installed using the package manager on Ubuntu Linux.

Update packages

sudo apt update

Install Docker

sudo apt install docker.io -y

Start Docker Service

sudo systemctl start docker

Enable Docker on boot

sudo systemctl enable docker

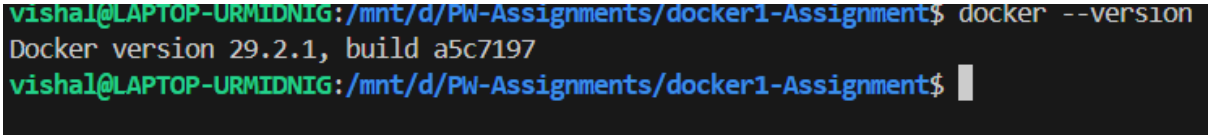
Verify Installation

docker --version

Example Output:

Docker version 24.x.x

📸 **Screenshot**



1.5 Running First Docker Container

To verify Docker installation, the **hello-world container** was executed.

sudo docker run hello-world

This command downloads the hello-world image from Docker Hub and runs it.

📸 **Screenshot**

```
vishal@LAPTOP-URMIDNIG:/mnt/d/PW-Assignments/docker1-Assignment$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

1.6 Creating a Dockerfile

A Dockerfile is a script containing instructions used to build Docker images.

File name:

Dockerfile

Example Dockerfile for containerizing a simple web application using Nginx:

FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

This Dockerfile performs the following tasks:

1. Pulls the Nginx base image
2. Copies a custom HTML file into the web server directory

📸 **Screenshot**

```
Docker1-Assignment > Dockerfile
1   FROM nginx:latest
2   COPY index.html /usr/share/nginx/html/index.html
```

1.7 Application Code

Create a simple HTML file.

File name:

index.html

Code:

<h1>Hello from Docker Container</h1>

📸 **Screenshot**

```
Docker1-Assignment > index.html > h1
1   <h1>Hello from Docker Container</h1>
```

.

1.8 Building Docker Image

Build the Docker image using:

docker build -t docker-webapp .

This command builds an image using the Dockerfile in the current directory.

📸 **Screenshot**

```
vishal@LAPTOP-URMIDNIG:/mnt/d/PW-Assignments/docker1-Assignment$ docker build -t docker-webapp .
[+] Building 2.1s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 104B
 => [internal] load metadata for docker.io/library/nginx:latest
 => [auth] library/nginx:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 73B
 => CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:bc45d248c4e1d1709321de61566eb2b64d4f0e32765239d66573666be7f13349
 => => resolve docker.io/library/nginx:latest@sha256:bc45d248c4e1d1709321de61566eb2b64d4f0e32765239d66573666be7f13349
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:364d5682fbb5b792c4482d51890991fb68460fa035323d7208ee6a685551c842
 => => exporting config sha256:b94b6d69246b423c9bafdda3ec826b1be8573ce14b99e225fcf1823be5f3c72d
 => => exporting attestation manifest sha256:d3c7858623768b8722260509ef441506793016d752a3878592520dc29f3a06a4
 => => exporting manifest list sha256:0c2b0bdf839cafbee7db033c270603b863c458fe8a153d151e34bc2dfc79e6cc
 => => naming to docker.io/library/docker-webapp:latest
 => => unpacking to docker.io/library/docker-webapp:latest
```

1.9 Running Docker Container

Run the container using:

docker run -d -p 8081:80 docker-webapp

Explanation:

Option	Purpose
-d	Run container in background
-p	Map container port to host
8081:80	Host port to container port

📸 **Screenshot**

```
vishal@LAPTOP-URMIDNIG:/mnt/d/PW-Assignments/docker1-Assignment$ docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS         PORTS                                       NAMES
cece6a7e48ef   docker-webapp   "/docker-entrypoint.…"   4 minutes ago   Up 4 minutes   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   amazing_gould
vishal@LAPTOP-URMIDNIG:/mnt/d/PW-Assignments/docker1-Assignment$
```

1.10 Verifying the Application

Open browser:

http://localhost:8081

Expected Output:

Hello from Docker Container

📸 **Screenshot**

.

1.11 Advantages of Docker

- Lightweight containers
- Faster application deployment
- Consistent environments
- Easy scaling
- Simplified dependency management

1.12 Conclusion

Docker simplifies application deployment by packaging applications and their dependencies into containers. Using Dockerfiles, developers can automate image creation and ensure consistent execution across environments.

In this assignment, Docker was installed successfully and a simple web application was containerized using a Dockerfile.